



FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

KITCHENLANG

A project submitted to
Mr. Jeneffer A. Sabonsolin
Computer Science Department
FEU Institute of Technology

In partial fulfillment of the requirements in
CSTPLANGS - TRANSLATION OF PROGRAMMING LANGUAGE
for 2nd Trimester, Academic Year 2025-2026

Escritor, Jude Keith Q.
Mangahas, Karl Stephen D.
Obligado, Jasmin Joyce F.

BSCSSE-TN33
03/25/2026

PROJECT OVERVIEW

- The project presents the design and implementation of a domain-specific language (DSL) tailored for modeling cooking processes within a structured “kitchen” environment. The language allows users to define dishes through recipes composed of ingredients, tools, and step-by-step techniques.
- The system simulates the fundamental phases of a compiler, including lexical analysis, syntax analysis, and semantic analysis, to process and validate cooking instructions. By transforming structured input into a parse tree representation, the project demonstrates how real-world processes can be formalized into programmable logic.

A. Purpose and Objectives

- To design a custom programming language with a clear and consistent syntax
- To implement core compiler components such as tokenization and parsing
- To model real-world workflows (cooking) into formal computational structures
- To enforce logical correctness through semantic validation

B. What this Project Encapsulates

★ Deep Understanding of Compiler Design

The project provides hands-on experience with:

- lexical analysis
- parsing techniques
- syntax trees
- semantic validation

★ Practical Application of Theory

Abstract concepts in programming languages are applied to a familiar domain, making them easier to understand and visualize.

★ Improved Problem-Solving Skills

Designing a language requires:

- structuring rules
- handling edge cases
- ensuring consistency

★ Strong Foundation for Advanced Systems

This project builds a foundation for:

- interpreters
- compilers
- domain-specific tools

★ Creative System Design

- By modeling cooking as a language, the project demonstrates how computational thinking can be applied creatively to non-traditional domains.

★ Implementation Skills

Developing the system in Python enhances:

- string processing
- data structures (trees)
- modular programming

C. Features

★ Domain-Specific Language (DSL) Engine

- Supports a custom-designed cooking language with structured syntax
- Allows definition of dishes, recipes, ingredients, tools, and steps
- Enforces grammar rules through parsing
- Performs semantic validation for logical correctness

★ Lexical and Syntax Analysis Module

- Tokenizes user input into keywords, identifiers, operators, and symbols
- Parses token streams using a structured grammar (CFG)
- Detects and reports syntax errors with meaningful messages
- Builds a structured internal representation (parse tree / AST)

★ Semantic Analysis Engine

- Validates correct usage of ingredients, tools, and steps
- Detects undeclared variables and invalid dependencies
- Enforces execution order and logical correctness
- Provides descriptive semantic error feedback

★ Parse Tree and AST Visualization

- Automatically generates a tree representation of the input program
- Displays hierarchical relationships between components
- Helps users understand how code is structured internally
- Can be visualized graphically within the interface

★ Derivation Generator

- Shows step-by-step derivation from grammar rules
- Demonstrates how input strings are derived from the start symbol (<kitchen>)
- Useful for learning parsing and formal grammar concepts
- Supports both partial and full derivations

★ Built-in Code Editor (Mini IDE)

Integrated coding environment for writing cooking programs. Features include:

- syntax highlighting
- line numbering
- auto-formatting
- error highlighting
- Allows real-time validation and execution

★ Drag-and-Drop Cooking Interface

Interactive UI where users can:

- drag ingredients
- combine components visually
- simulate cooking steps

★ **Dual Interaction Mode**

Users can choose between:

- Code Mode - write DSL programs manually
- Visual Mode - construct recipes using drag-and-drop

Both modes are synchronized:

- visual actions generate code
- code updates reflect visually

★ **Cooking Simulation Engine**

- Simulates execution of steps in order
- Tracks transformation of ingredients into final dish
- Displays intermediate outputs (e.g., chopped → mixed → cooked)

★ **Error Detection and Feedback System**

Real-time detection of:

- syntax errors
- semantic violations

Provides:

- clear error messages
- line references

★ **Execution Trace Viewer**

- Displays step-by-step execution flow
- Shows how each instruction transforms data
- Helps users debug and understand program behavior

★ **Code-to-Tree and Tree-to-Code Conversion**

- Converts written code into tree structures
- Reinforces understanding of parsing and structure

D. Keywords

- dish - defines final output
- recipe - defines a recipe block
- ingredient - declares ingredients
- step - defines a cooking step
- use - declares tools
- mix, chop, fry, boil, bake - techniques
- with - connects elements
- into - output/result transformation
- serve - final output

E. Identifiers

- Must start with a letter
- Can contain letters, numbers, underscores
- Case-sensitive

Examples:

- chicken_adobo
- pan1
- rice
- mainDish

F. Literals

- types:
- Numbers -> quantities
- Strings -> names/descriptions

G. Operators

- ☐ = → assignment
- ☐ + → combine ingredients

H. Symbols

- ☐ , → separator
- ☐ ; → statement terminator
- ☐ () → grouping
- ☐ {} → blocks

I. Comment

- # this is a comment for the project

J. Program Structure

→ <kitchen> ::= <dish_def>

K. Dish Declaration

→ <dish_def> ::= dish_def <identifier> { <recipe_mk> <serve_out> }

L. Recipe Declaration

→ <recipe_mk> ::= recipe_mk { <ingredient_def> <tool_use> <step_list> }

M. Ingredient Declaration

→ <ingredient_def> ::= ingredient_def <ingredient_list> ;

→ <ingredient_list> ::= <ingredient_item>
| <ingredient_item> , <ingredient_list>

→ <ingredient_item> ::= <identifier> = <number>

N. Tool Declaration

→ <tool_use> ::= tool_use <tool_list> ;

→ <tool_list> ::= <identifier>
| <identifier> , <tool_list>

O. Step List:

→ <step_list> ::= <step_do>
| <step_do> <step_list>

P. Step Declaration

→ <step_do> ::= step_do <action_stmt> ;

Q. Action Statement

→ <action_stmt> ::= <technique> (<arg_list>) = <identifier>

R. Technique

→ <technique> ::= mix | chop | fry | boil | bake | saute | blanche | stir | toss

S. Argument List

→ <arg_list> ::= <identifier>
| <identifier> , <arg_list>

T. Serve (Output)

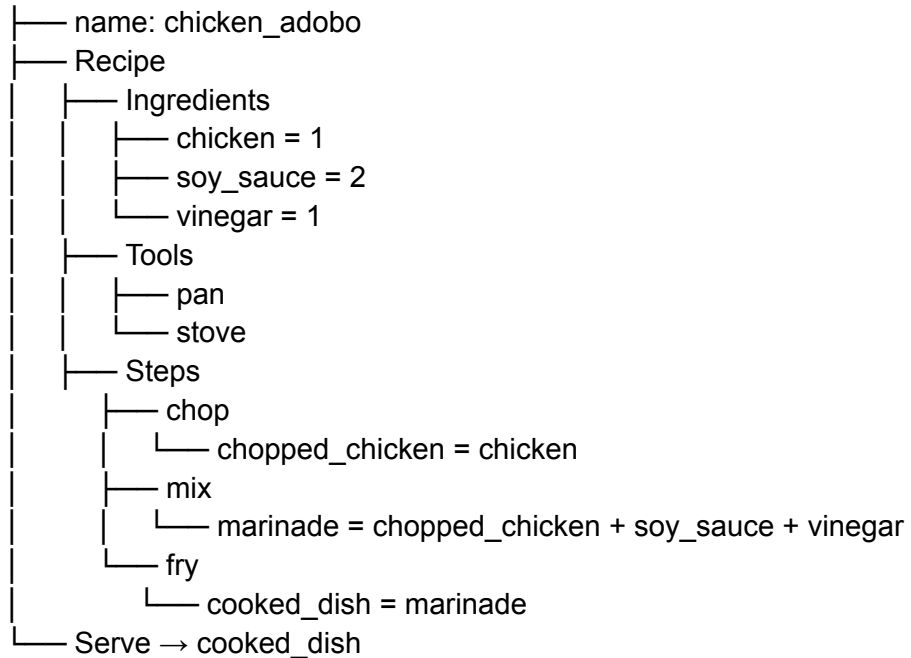
→ <serve_out> ::= serve_out <identifier> ;

U. Sample Program:

```
dish_def chicken_adobo {  
  recipe_mk {  
    ingredient_def chicken = 1, soy_sauce = 2, vinegar = 1;  
  
    tool_use pan, stove;  
  
    step_do chop {  
      chopped_chicken = chicken;  
    }  
  
    step_do mix {  
      marinade = chopped_chicken + soy_sauce + vinegar;  
    }  
  
    step_do fry {  
      cooked_dish = marinade;  
    }  
  }  
  
  serve_out cooked_dish;  
}
```

V. Syntax Tree Sample:

Dish



W. Derivation Example:

Target:

step_do chop { chopped_chicken = chicken; }

<step_do>

- step_do <technique> { <assignment_stmt> }
- step_do chop { <assignment_stmt> }
- step_do chop { <identifier> = <expression> ; }
- step_do chop { chopped_chicken = chicken ; }

X. Semantic Rules:

- Ingredients must be declared before use
- Tools must be declared
- Step outputs must exist before being reused
- Serve must reference final output

1. Ingredient Declaration

All ingredients must be declared before they are used in any step.

Valid

- `ingredient_def chicken = 1;`
- `step_do chop(chicken) = chopped_chicken;`

Invalid:

- `step_do chop(beef) = chopped_beef; // beef not declared`

2. Tool Declaration Rule

All tools required by a technique must be declared beforehand.

Valid

- `tool_use pan;`
- `step_do fry(chicken) = fried_chicken;`

Invalid

- `step_do fry(chicken) = fried_chicken; // no pan/stove declared`

3. Step Dependency Rule

Outputs from steps must exist before they are reused in later steps.

Valid

```
step_do chop(chicken) = chopped_chicken;  
step_do mix(chopped_chicken, soy_sauce) = marinade;
```

Invalid

```
ingredient_def chicken = 1;  
ingredient_def chicken = 2; // duplicate declaration
```

4. Type Consistency Rule

Only valid types must be used in operations.

Ingredients - numeric quantities

Step outputs - derived items

Invalid:

```
ingredient_def chicken = "one"; // invalid type
```

5. Final Output Rule

The identifier in `serve_out` must correspond to a valid, previously computed step output.

Valid:

```
serve_out cooked_dish;
```

Invalid

```
serve_out raw_chicken; // not a final processed output
```

6. Order of Execution Rule

Steps must follow a logical sequence where dependencies are resolved in order.

Invalid

```
step_do fry(marinade) = cooked;  
step_do mix(chicken, sauce) = marinade;
```

7. Tool-Technique Compatibility Rule

Certain techniques require specific tools.

Examples:

chop - requires knife

fry - requires pan/stove

boil - requires pot

8. Minimum Structure Rule

- A valid kitchen program must contain:
- at least one dish
- one recipe
- one ingredient
- one step
- one serve statement

Y. Error Examples:

1. Syntax errors (Structure issues):

- `ingredient_def chicken = 1` // missing semicolon
- `dish_def adobo` // missing braces
- `step_do chop chicken = x;` // missing parentheses

2. Semantic Errors (Logical Issues):

- ```
step_do fry {
 cooked = raw_meat;
} // raw_meat undeclared
```
- ```
serve_out dish; // not produced by any step
```
- ```
ingredient_def chicken = 1;
ingredient_def chicken = 2; // duplicate
```
- ```
step_do boil(chicken) = cooked; // no pot declared
```

Z. Token Definitions

- Keywords cannot be used as identifiers
- Identifiers must follow naming rules (start with letter)
- Numbers must be integers only
- Comments are ignored during tokenization

Token Type	Description	Examples
KEYWORD	Reserved words with fixed meaning	dish_def, recipe_mk, ingredient_def
IDENTIFIER	User-defined names	chicken, pan1, marinade
NUMBER	Integer values for quantities	1, 2, 10
OPERATOR	Symbols for operations	=, , +
SYMBOL	Structural characters	{ } () ; ,